



QuickCanteen

How the food-ordering website and Android app were built

Live site: <https://campus-canteen-live.vercel.app> · Code: github.com/sreevathson1530/campus-canteen-live

Built 23 to 30 September 2026 · 27 commits · 285 automated tests

1. What it is

QuickCanteen lets customers order food from their phone, confirm the order with a 4-digit code sent to their email, watch the order move from **Placed** to **Preparing** to **Ready** in real time, and collect it at the counter with a token number. The kitchen sees every order the moment it is placed. The owner gets an admin panel with live orders, earnings, the menu, stock, bills and users. When an order is collected, a bill is saved and emailed to the customer.

It has three kinds of user, each with its own screens:

Role	Signs in with	What they can do
Customer	Their own email, registered on the site	Browse the menu, add to cart, confirm with a code, track the order, see past orders and bills, order again
Kitchen	<code>kitchen@canteen.cit</code>	See incoming orders, tap Start, Ready and Collect, reject an order, manage stock, open or close the canteen
Admin	<code>admin@canteen.cit</code>	Everything the kitchen can do, plus the dashboard, all orders with earnings, menu editing, bills, users and settings

2. The technology used

Part	Technology	Why
Website and server	Next.js 16, React 19, TypeScript	One project holds both the pages and the server code, so there is one thing to deploy
Styling	Tailwind CSS 4, shadcn/ui components	Consistent design built from a small set of colour and spacing rules
Database	PostgreSQL on Neon (SQLite on the developer's PC), accessed with Prisma 6	Reliable storage with transactions, which matter for stock and token numbers
Live updates	WebSockets, with Upstash Redis passing messages between servers	Orders appear on the kitchen screen and the customer's tracker without refreshing
Login	Passwords hashed with bcrypt; a signed session cookie (JWT) lasting 7 days	Passwords are never stored in readable form
Email	Gmail's mail server through nodemailer	Free, needs no domain, delivers to any address (about 500 emails a day)
Hosting	Vercel (Singapore region)	Free tier; every push to GitHub deploys automatically
Android app	Capacitor, built by GitHub Actions	Wraps the live site in an installable app without an app-store fee
Tests	Vitest (unit and integration), Playwright (real browser)	Catches mistakes before they reach the live site

3. How the pieces fit together

Customer phone · Kitchen phone · Admin laptop · Android app
all open the same website

Vercel

Runs the pages and 30 server endpoints
(/api/...)

Neon Postgres

Users, menu, orders, bills, settings

Upstash Redis

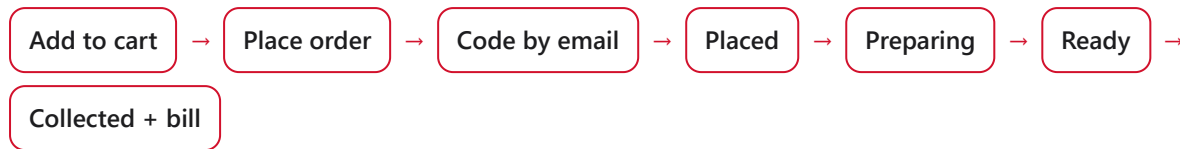
Relays live events between server instances; rate limits

Gmail

Sends order codes and bills

The backend is not a separate program. It lives in the same project: the endpoints are in `src/app/api/`, the rules behind them in `src/lib/`, and the database structure in `prisma/`.

4. How an order works, step by step



1. **Cart.** The cart is kept on the customer's phone. Nothing is reserved until the order is placed.
2. **Code.** Tapping Place order sends a random 4-digit code to the customer's email. The code lasts 5 minutes, allows 5 wrong tries, and can be re-sent after 30 seconds. This stops prank orders.
3. **Placing.** Once the code is correct, the server does everything in one database transaction: it checks the canteen is open, checks each dish is available, takes stock, gives the next token number for the day (starting at 101), and saves the order. If anything fails, nothing is saved.
4. **Kitchen.** The order appears instantly on the kitchen board. Staff tap **Start**, then **Ready**. The customer's phone updates each time and plays a chime at Ready.
5. **Collection.** Staff type the token number and tap **Collect**. In the same transaction a bill is created as a permanent copy of the order (name, mobile, dishes, prices, total, times). The bill is then emailed to the customer.

Why the order is safe against mistakes. Stock can never go below zero even if ten people order the last three items at the same moment: exactly three succeed. A double tap on Place order creates one order, not two. Two staff tapping the same order at once cannot both succeed; the second is told it was already updated.

5. How it was built, in order

#	Stage	What was done
1	Plan	Read the requirements document, wrote a design and an implementation plan, and got approval before writing code. Decisions agreed up front: mobile-first for every role, no counter TV, a code before every order, a bill on collection.
2	Foundation	Project set up with the database structure, login and registration, and the live connection.
3	Ordering	Menu, cart, order placement with stock and token numbers, the order tracker and the list of past orders.
4	Kitchen and admin	Kitchen board with Start, Ready and Collect; stock screen; admin dashboard, menu editor, users, settings and bills.
5	Going online	Deployed to Vercel with Neon and Upstash. The live-update system was rewritten for Vercel (see section 8).
6	Richer content	Public home page, menu search and filters, dish details (spice, calories, ingredients, allergens, "goes well with"), Order again, dashboard trends.
7	Codes by email	Order codes moved from SMS to email through Gmail, because free SMS needed an Android phone kept online.
8	Admin orders and bills by email	Admin → Orders with earnings for today, this week and this month; bills emailed on collection with a Resend button.
9	Redesign and new menu	The look was rebuilt in the style of mainstream food-ordering sites and the menu replaced with 30 fast-food and café items.
10	App	Made installable from the browser, then packaged as an Android APK built in the cloud.
11	Fixes	Footer removed, staff logins renamed to <code>@canteen.cit</code> , and pages that ran wider than a phone screen fixed.

6. What is stored in the database

Table	What it holds
User	Name, email, mobile number, hashed password and role. One account per email and per mobile number.
Category, MenuItem	The menu: name, price, photo, veg or non-veg, stock, prep time, spice level, calories, ingredients, allergens, badges and pairings. Removed dishes are archived, not deleted, so old orders still show them.
Order, OrderItem	Each order with its token, status, total and times; each line copies the dish name and price at the moment of ordering, so later price changes do not alter past orders.
DailyCounter	The last token number given out each day.
OtpChallenge	Order codes, stored only as a keyed hash, with expiry and attempt count.
Bill, BillLine	The permanent bill made at collection, and when it was emailed.
Settings	Name, open or closed, closed message, minutes per order, opening hours and location.

Money is stored in paise (₹50 is saved as 5000) to avoid rounding errors. Times are stored in UTC and shown in Indian time.

7. Security and abuse protection

- Passwords are hashed with bcrypt. Order codes and tokens are stored only as keyed hashes, so a database leak does not reveal usable codes.
- The session cookie cannot be read by page scripts (`httpOnly`) and every server endpoint checks the user's role itself.
- Limits: 5 codes an hour per mobile number, 60 an hour per network address, and a 30-second wait between codes. Login and registration are rate-limited too.
- Kitchen staff see a customer's first name and a masked phone number only. The full number is visible to the customer and the admin.
- Passwords and keys are never in the code. They are kept as hidden settings on Vercel and as private secrets on GitHub.

8. How live updates work

Each open page keeps a WebSocket connection to the server and is placed in "rooms": everyone is in `public`, each customer is in a private room, staff are in `kitchen`, admins in `admin`. When something changes, the server sends an event only to the rooms that should see it, and only after the database has saved the change.

On Vercel, requests are handled by many short-lived server instances, and a connection is tied to one of them. So every event is first published to Redis, and every instance forwards it to its own connected phones. If a phone loses its connection, it reconnects by itself (waiting 1 to 30 seconds) and reloads the latest data, so nothing is missed.

9. The design journey

The look went through three versions, guided by feedback:

1. **First version:** a warm "banana leaf and turmeric" theme with a heavy display font and interactive 3D models of each dish built in Three.js.
2. **Simplified:** the 3D views and heavy styling were removed when the request became "simple, no more 3D".
3. **Final:** rebuilt in the style of Domino's and Thalappakatti-type ordering sites: a red brand bar, a light page, one typeface, category buttons with photos that follow the scroll, dish rows with a photo and an **ADD** button that becomes a $- 1 +$ counter, and a cart bar at the bottom. On larger screens the rows become a grid of cards.

The menu was replaced at the same time: pizzas, burgers, sides, wraps and sandwiches, coffee and drinks, and desserts. All 30 photos are freely licensed images from Wikimedia Commons, chosen by eye to avoid brand logos, cropped to one shape, and credited on the site's Credits page as their licences require.

10. Putting it online

- The code is on GitHub. Every push to the `main` branch makes Vercel build and publish a new version automatically, usually in about two minutes.
- During each build the database is brought up to date: structure changes are applied, and the starter data (staff accounts and the menu) is created if missing. Existing data and any changes made in the admin panel are never overwritten.
- The database (Neon) and Redis (Upstash) were added from Vercel's marketplace on their free plans, in the same Singapore region as the server to keep it fast.
- Email uses a Gmail account with an **app password**, a separate 16-letter password made only for this app. It is saved on Vercel as a hidden setting.

11. The Android app

There are two ways to have it as an app, and both show the same live site:

- **Install from the browser.** The site describes itself as an app (name, icon, colours), so Chrome offers "Install app" and iPhone's Safari offers "Add to Home Screen".
- **Android APK.** A small Capacitor project in the `mobile/` folder wraps the site. This PC did not have enough free memory to build it, so GitHub builds it in the cloud, signs it with a private key, and publishes it as a download. Rebuilds happen automatically when the `mobile/` folder changes.

The signing key matters. It is kept in `C:\Users\sreev\Projects\campus-canteen-keys\` and as private GitHub secrets. Every future update of the app must be signed with the same key, so keep a private backup and never share it.

Because the app opens the live site, website updates appear in the app straight away with nothing to reinstall. It needs an internet connection.

12. How it was tested

Kind	What it checks
Unit tests	Small rules on their own: input checking, money and time formatting, which status changes are allowed (222 cases), email masking and content.
Integration tests	Real database operations: ten simultaneous orders on three items of stock, double-tap protection, code limits and expiry, bills, earnings, search, and the live-update rooms.
Same tests on PostgreSQL	The integration tests are run again on a real PostgreSQL database, the kind used in production.
Browser tests	A real Chrome plays a customer and a kitchen at once through a whole order, and checks every page for every role fits a 360-pixel phone screen.
Live checks	After each deploy the live site was checked directly: pages, logins, the menu, and whether it can sign in to Gmail.

There are 285 unit and integration tests and 4 browser tests. They are run before each change is published.

13. Problems met and how they were solved

Problem	What fixed it
Live updates did not work on Vercel	The first version used Socket.IO on one long-running server. Vercel has no such server, so it was rewritten with plain WebSockets and Redis passing events between instances.
Order codes never arrived	The site was in test mode, printing codes to the log. Free SMS needed a phone kept online, and free email services need an owned domain. Gmail's own mail server solved both.
Gmail rejected the first password	The app password was not accepted for the account. A new one was created and tested before being saved.
Editing one field of a dish reset its stock	A quirk in the input-checking library re-applied default values on partial updates. The update rule was rebuilt without defaults, with a test to keep it fixed.
Pages wider than the phone screen	Found twice. First, hidden accessibility labels inside a side-scrolling row escaped its edges. Later, three admin pages stretched to their widest row. Both were fixed at the cause, and a test now checks every page.
The PC ran out of memory	Builds and browser tests crashed when memory was full. Heavy work (the Android build) was moved to GitHub's servers.
Wrong photos from search	Automatic image search returned packaging, logos and the wrong dishes. Each photo was reviewed by eye and replaced where needed.

14. Where things are, and how to change them

To do this	Go here
Change a dish, price, photo or stock	Sign in as admin → Menu (or Kitchen → Stock). No code change needed.
Mark an order ready or collected	Kitchen screen: Start → Ready → Collect.
See all orders and earnings	Admin → Orders.
Open or close the canteen, change hours	Kitchen → Stock, or Admin → Settings.
Look at the raw data	vercel.com → the project → Storage → Neon → Tables.
See server errors	vercel.com → the project → Logs.
Change how pages look	<code>src/components/</code> (screens) and <code>src/app/globals.css</code> (colours).
Change server rules	<code>src/lib/</code> , for example <code>orders/service.ts</code> .
Read why each decision was made	<code>DECISIONS.md</code> in the project.

Passwords are deliberately not printed in this document. Keep them somewhere private.